```
!pip install torch_geometric
```

Gizli çıkışı göster

```
!pip install rdkit
```

```
Collecting rdkit
  Downloading rdkit-2026.3.1-cp312-cp312-manylinux_2_28_x86_64.whl.met
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dis
Downloading rdkit-2026.3.1-cp312-cp312-manylinux_2_28_x86_64.whl (37.0
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 37.0/37.0 MB 13.9 MB/s eta
Installing collected packages: rdkit
Successfully installed rdkit-2026.3.1
```

```
# ---------------------- 1. İTHALATLAR VE SABİTLER ----------------------

import random
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import (
    roc_auc_score, roc_curve, auc,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.model_selection import StratifiedKFold
from scipy.stats import ttest_rel, wilcoxon

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import WeightedRandomSampler
from torch_geometric.loader import DataLoader

from torch_geometric.data import Data, Dataset
from torch_geometric.nn import GINConv, global_mean_pool
from torch_geometric.utils import add_self_loops

from rdkit import Chem
from rdkit.Chem import Descriptors

# Sabitler
SEED           = 42
BATCH_SIZE     = 128
LR_BASE        = 1e-4
WEIGHT_DECAY   = 1e-3
EPOCHS         = 60
PATIENCE       = 8
HIDDEN_DIM     = 128
N_LAYERS       = 4
DROPOUT_RATE   = 0.2
DEVICE         = torch.device("cuda" if torch.cuda.is_available() else "cpu")
FCL_ALPHA      = 0.25
FCL_GAMMA      = 2.0
```

```
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
warnings.filterwarnings("ignore", category=UserWarning)

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(SEED)
```

```
# ---------------------- 2. FEATURIZER VE SMILES→Data ----------------------

class Featurizer:
    def __init__(self, allow_map):
        self.dim, self.map = 0, {}
        for key, vals in allow_map.items():
            sorted_vals = sorted(list(vals))
            self.map[key] = {v: i + self.dim for i, v in enumerate(sorted_vals)}
            self.dim += len(sorted_vals)

    def _encode(self, obj):
        vec = np.zeros(self.dim, dtype=np.float32)
        for method_name, mapping in self.map.items():
            val = getattr(self, method_name)(obj)
            if val in mapping:
                vec[mapping[val]] = 1.0
        return vec

class AtomFeat(Featurizer):
    def n_number(self, atom):            return atom.GetAtomicNum()
    def n_valence(self, atom):           return atom.GetTotalValence()
    def n_hydrogens(self, atom):         return atom.GetTotalNumHs()
    def hybridization(self, atom):       return atom.GetHybridization().name.lowe
    def n_degree(self, atom):            return atom.GetDegree()
    def n_radical_electrons(self, atom): return atom.GetNumRadicalElectrons()
    def is_in_ring(self, atom):          return atom.IsInRing()
    def encode(self, atom):              return self._encode(atom)

class BondFeat(Featurizer):
    def __init__(self, allow_map):
        super().__init__(allow_map)
        self.dim += 1

    def bond_type(self, bond):      return bond.GetBondType().name.lower()
    def conjugated(self, bond):     return bond.GetIsConjugated()

    def encode(self, bond):
        if bond is None:
            vec = np.zeros(self.dim, dtype=np.float32)
            vec[-1] = 1.0
            return vec
        return self._encode(bond)
```

```
# Atom ve bond öznitelik haritaları
ATOM_FTR = AtomFeat({
    "n_number":           {1, 5, 6, 8, 9, 11, 15, 16, 17, 20, 35, 53},
    "n_valence":          {0, 1, 2, 3, 4, 5, 6},
    "n_hydrogens":        {0, 1, 2, 3, 4},
    "hybridization":      {"s", "sp", "sp2", "sp3"},
    "n_degree":           {0, 1, 2, 3, 4, 5, 6},
    "n_radical_electrons":{0, 1, 2, 3, 4},
    "is_in_ring":         {True, False}
})
BOND_FTR = BondFeat({
    "bond_type":  {"single", "double", "triple", "aromatic"},
    "conjugated": {True, False}
})

def mol_from_smiles(smiles, randomize=False):
    """
    RDKit ile SMILES → Mol dönüştürür. Eğer randomize=True ise,
    MolToSmiles(doRandom=True) yöntemiyle rastgele bir düzenleme yapar.
    """
    if randomize:
        base = Chem.MolFromSmiles(smiles)
        smiles = Chem.MolToSmiles(base, doRandom=True)
    mol = Chem.MolFromSmiles(smiles, sanitize=False)
    flag = Chem.SanitizeMol(mol, catchErrors=True)
    if flag != Chem.SanitizeFlags.SANITIZE_NONE:
        Chem.SanitizeMol(mol, Chem.SanitizeFlags.SANITIZE_ALL ^ flag)
    Chem.AssignStereochemistry(mol, cleanIt=True, force=True)
    return mol

def pyg_from_mol(mol, y=None):
    """
    RDKit Mol → PyG Data objesine çevirir:
    - Atomlar için ATOM_FTR.encode ile özellik vektörü
    - Bağlar için BOND_FTR.encode
    """
    x = torch.from_numpy(
        np.stack([ATOM_FTR.encode(atom) for atom in mol.GetAtoms()])
    ).float()

    edge_index_list, edge_attr_list = [], []
    num_atoms = mol.GetNumAtoms()

    # Self-loop ekle (bond=None)
    for i in range(num_atoms):
        edge_index_list.append([i, i])
        edge_attr_list.append(BOND_FTR.encode(None))

    # Gerçek bağlar
    for bond in mol.GetBonds():
        i = bond.GetBeginAtomIdx()
        j = bond.GetEndAtomIdx()
        f = BOND_FTR.encode(bond)
        edge_index_list += [[i, j], [j, i]]
        edge_attr_list += [f, f]

    edge_index = torch.tensor(edge_index_list, dtype=torch.long).t().contiguous()
    edge_attr = torch.tensor(np.stack(edge_attr_list), dtype=torch.float)

    data = Data(x=x, edge_index=edge_index, edge_attr=edge_attr)
```

```
    if y is not None:
        data.y = torch.tensor([y], dtype=torch.float)
    return data

def compute_descriptors(smiles):
    """
    RDKit kullanarak molekül düzeyinde 6 boyutlu descriptor çıkarır:
    MolLogP, MolWt, TPSA, NumHAcceptors, NumHDonors, NumRotatableBonds
    """
    m = Chem.MolFromSmiles(smiles)
    descs = [
        Descriptors.MolLogP(m),
        Descriptors.MolWt(m),
        Descriptors.TPSA(m),
        Descriptors.NumHAcceptors(m),
        Descriptors.NumHDonors(m),
        Descriptors.NumRotatableBonds(m)
    ]
    return np.array(descs, dtype=np.float32)

def add_virtual_node(data):
    """
    Her grafın sonuna bir "virtual node" ekler. Bu düğüm
    yalnızca global descriptor bilgisini aktarmak için kullanılır.
    """
    num_nodes = data.x.size(0)
    feat_dim = data.x.size(1)

    # Sanal düğüm feature: sıfır vektör (global_desc model içinde kullanılıyor)
    vn_feat = torch.zeros((1, feat_dim), device=data.x.device)
    data.x = torch.cat([data.x, vn_feat], dim=0)

    # Batch tensor'una ekle
    data.batch = torch.cat([data.batch, data.batch.new_full((1,), data.batch[0])]

    # Sanal düğüm index'i
    vs = num_nodes
    existing = torch.arange(0, num_nodes, device=data.x.device)

    # Sanal → her atom (kolektif bağlantı)
    e1 = torch.stack([torch.full((num_nodes,), vs, device=data.x.device), existin
    # Her atom → sanal
    e2 = torch.stack([existing, torch.full((num_nodes,), vs, device=data.x.device

    data.edge_index = torch.cat([data.edge_index, e1, e2], dim=1)

    # Kenar özniteliği ekle (bond_dim = mevcut bond öznitelik sayısı)
    bond_dim = data.edge_attr.size(1)
    new_edge_attr = torch.zeros((num_nodes * 2, bond_dim), device=data.edge_attr.
    data.edge_attr = torch.cat([data.edge_attr, new_edge_attr], dim=0)

    return data
```

```
# ---------------------- 3. DATASET ----------------------

class HIVDatasetVN(Dataset):
    """
    Her satır için:
```

```
    - SMILES → Mol → pyg_from_mol ile PyG Data objesi
    - data.global_desc: [1×6] tensor olarak descriptor bilgisi
    - Eğer virtual_node=True ise add_virtual_node(data) çağrılır
    """
    def __init__(self, df, augment=False, virtual_node=False):
        super().__init__()
        self.df = df.reset_index(drop=True)
        self.aug = augment
        self.virtual_node = virtual_node

    def len(self):
        return len(self.df)

    def get(self, idx):
        row = self.df.iloc[idx]
        smiles = row.smiles
        mol = mol_from_smiles(smiles, self.aug)
        data = pyg_from_mol(mol, row.HIV_active)

        # Descriptor'ı 1×6 tensor olarak ekle
        desc = compute_descriptors(smiles)                                  # shape
        data.global_desc = torch.tensor(desc, dtype=torch.float).unsqueeze(0)  #

        # Batch bilgisi: her atom için 0 (tek graf)
        data.batch = torch.tensor([0] * data.x.size(0), dtype=torch.long)

        if self.virtual_node:
            data = add_virtual_node(data)

        return data
```

```
# ----------------------- 4. FOCAL LOSS -----------------------

from torchvision.ops import sigmoid_focal_loss

def focal_loss(inputs, targets, alpha=FCL_ALPHA, gamma=FCL_GAMMA):
    """
    Sigmoid Focal Loss uygular.
    - inputs: logits (batch_size)
    - targets: 0/1 etiketleri (batch_size)
    """
    return sigmoid_focal_loss(inputs, targets, alpha=alpha, gamma=gamma, reductio

class GIN_MPNN(nn.Module):
    def __init__(self, in_dim, hidden_dim=HIDDEN_DIM, n_layers=N_LAYERS,
                 dropout=DROPOUT_RATE, desc_dim=0):
        super().__init__()
        self.convs = nn.ModuleList()
        self.bns   = nn.ModuleList()
        self.dropout = nn.Dropout(dropout)

        # İlk GINConv katmanı (in_dim → hidden_dim)
        mlp1 = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim)
        )
        self.convs.append(GINConv(mlp1))
```

```
        self.bns.append(nn.BatchNorm1d(hidden_dim))

        # Diğer GIN katmanları (hidden_dim → hidden_dim)
        for _ in range(n_layers - 1):
            mlpi = nn.Sequential(
                nn.Linear(hidden_dim, hidden_dim),
                nn.ReLU(),
                nn.Linear(hidden_dim, hidden_dim)
            )
            self.convs.append(GINConv(mlpi))
            self.bns.append(nn.BatchNorm1d(hidden_dim))

        self.read = global_mean_pool

        # Descriptor projeksiyon bloğu (desc_dim → hidden_dim//4)
        if desc_dim > 0:
            self.desc_proj = nn.Sequential(
                nn.Linear(desc_dim, hidden_dim // 2),
                nn.ReLU(),
                nn.Dropout(dropout),
                nn.Linear(hidden_dim // 2, hidden_dim // 4),
                nn.ReLU()
            )
            total_emb_dim = hidden_dim + hidden_dim // 4
        else:
            self.desc_proj = None
            total_emb_dim = hidden_dim

        # Son MLP: total_emb_dim → total_emb_dim//2 → total_emb_dim//4 → 1
        self.mlp = nn.Sequential(
            nn.Linear(total_emb_dim, total_emb_dim // 2),
            nn.BatchNorm1d(total_emb_dim // 2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(total_emb_dim // 2, total_emb_dim // 4),
            nn.BatchNorm1d(total_emb_dim // 4),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(total_emb_dim // 4, 1)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        h = x
        for conv, bn in zip(self.convs, self.bns):
            h = conv(h, edge_index)
            h = bn(h)
            h = F.relu(h)
            h = self.dropout(h)

        # Global mean pooling → graph seviyesi vektör (batch_size×hidden_dim)
        g = self.read(h, batch)

        # Descriptor projeksiyonu varsa, concat et
        if self.desc_proj is not None:
            # data.global_desc: shape (batch_size × desc_dim)
            d = self.desc_proj(data.global_desc.to(DEVICE))
            g = torch.cat([g, d], dim=1)

        out = self.mlp(g).view(-1)
```

```
        return out
```

```
# ---------------------- 6. EĞİTİM VE DOĞRULAMA FONKSİYONLARI -----------------

def run_epoch_onecycle(loader, model, crit, optimizer, scheduler):
    """
    Her batch sonunda scheduler.step() çağrılan OneCycleLR protokolü ile bir epoc
    """
    model.train()
    total_loss, true_list, pred_list = 0.0, [], []

    for batch in loader:
        batch = batch.to(DEVICE)
        logits = model(batch)
        loss = crit(logits, batch.y)

        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 0.5)
        optimizer.step()
        scheduler.step()

        total_loss += loss.item() * batch.num_graphs
        true_list.append(batch.y.detach().cpu().numpy())
        pred_list.append(torch.sigmoid(logits.detach()).cpu().numpy())

    true_all = np.concatenate(true_list)
    pred_all = np.concatenate(pred_list)
    auc_score = roc_auc_score(true_all, pred_all)
    return total_loss / len(loader.dataset), auc_score

def run_validation(loader, model, crit, threshold=0.5):
    """
    Validation veya test aşaması:
    loss + Accuracy + Precision + Recall + F1 + ROC-AUC döndürür.
    """
    model.eval()
    total_loss, true_list, pred_list = 0.0, [], []

    with torch.no_grad():
        for batch in loader:
            batch = batch.to(DEVICE)
            logits = model(batch)
            loss = crit(logits, batch.y)

            total_loss += loss.item() * batch.num_graphs
            true_list.append(batch.y.detach().cpu().numpy())
            pred_list.append(torch.sigmoid(logits).detach().cpu().numpy())

    true_all = np.concatenate(true_list)
    pred_all = np.concatenate(pred_list)
    pred_bin = (pred_all >= threshold).astype(int)

    metrics = {
        "loss": total_loss / len(loader.dataset),
        "accuracy": accuracy_score(true_all, pred_bin),
        "precision": precision_score(true_all, pred_bin, zero_division=0),
        "recall": recall_score(true_all, pred_bin, zero_division=0),
```

```
            "f1": f1_score(true_all, pred_bin, zero_division=0),
            "roc_auc": roc_auc_score(true_all, pred_all)
        }

        return metrics, true_all, pred_all

    def train_onecycle(train_loader, val_loader, sample_data, desc_dim):
        """
        Modeli OneCycleLR scheduler ile eğitir, her epoch sonunda validation yapar.
        """
        model = GIN_MPNN(
            in_dim=sample_data.x.size(1),
            hidden_dim=HIDDEN_DIM,
            n_layers=N_LAYERS,
            dropout=DROPOUT_RATE,
            desc_dim=desc_dim
        ).to(DEVICE)

        crit = focal_loss
        optimizer = torch.optim.AdamW(model.parameters(), lr=LR_BASE, weight_decay=WE

        total_steps = EPOCHS * len(train_loader)
        scheduler = torch.optim.lr_scheduler.OneCycleLR(
            optimizer,
            max_lr=LR_BASE,
            total_steps=total_steps,
            pct_start=0.1,
            anneal_strategy='cos',
            final_div_factor=10
        )

        best_val_auc = 0.0
        wait = 0
        history = {'tr_auc': [], 'va_auc': [], 'tr_ls': [], 'va_ls': []}

        for epoch in range(1, EPOCHS + 1):
            tr_loss, tr_auc = run_epoch_onecycle(train_loader, model, crit, optimizer
            val_metrics, _, _ = run_validation(val_loader, model, crit)

            va_loss = val_metrics["loss"]
            va_auc = val_metrics["roc_auc"]

            history['tr_ls'].append(tr_loss)
            history['va_ls'].append(va_loss)
            history['tr_auc'].append(tr_auc)
            history['va_auc'].append(va_auc)

            print(
                f"Epoch {epoch:02d}  "
                f"Loss: {tr_loss:.4f}/{va_loss:.4f}  "
                f"AUC: {tr_auc:.3f}/{va_auc:.3f}  "
                f"Gap: {tr_auc - va_auc:.3f}"
            )

            if va_auc > best_val_auc + 1e-4:
                best_val_auc = va_auc
                wait = 0
                torch.save(model.state_dict(), "best_model_onecycle.pt")
            else:
                wait += 1
                if wait >= PATIENCE:
```

```
                print("Early stopping tetiklendi.")
                break

    model.load_state_dict(torch.load("best_model_onecycle.pt"))
    return model, history, best_val_auc
```

```
# ---------------------- 7. CROSS-VALIDATION VE EĞİTİM ----------------------

def cross_validate_training(df, k_folds=5):
    """
    Stratified K-Fold cross-validation:
    - Her fold için train/val/test ayrımı yapar.
    - Fold bazlı Accuracy, Precision, Recall, F1, ROC-AUC üretir.
    - Mean ± std ve variance hesaplar.
    """
    skf = StratifiedKFold(n_splits=k_folds, shuffle=True, random_state=SEED)

    all_histories = []
    fold_results = []

    for fold, (train_idx, temp_idx) in enumerate(
            skf.split(df.smiles, df.HIV_active), start=1):

        print(f"\n{'='*25} FOLD {fold} {'='*25}")
        set_seed(SEED + fold)

        df_train = df.iloc[train_idx].reset_index(drop=True)
        df_temp  = df.iloc[temp_idx].reset_index(drop=True)

        df_temp = df_temp.sample(frac=1.0, random_state=SEED + fold).reset_index(
        half = len(df_temp) // 2
        df_val = df_temp[:half].reset_index(drop=True)
        df_test = df_temp[half:].reset_index(drop=True)

        print(f"Train: {len(df_train)}, Val: {len(df_val)}, Test: {len(df_test)}"

        train_ds = HIVDatasetVN(df_train, augment=True, virtual_node=True)
        val_ds   = HIVDatasetVN(df_val, augment=False, virtual_node=True)
        test_ds  = HIVDatasetVN(df_test, augment=False, virtual_node=True)

        labels_train = df_train.HIV_active.values
        class_counts = np.array([len(np.where(labels_train == t)[0]) for t in np.
        weights = 1.0 / class_counts
        samples_weight = np.array([weights[int(t)] for t in labels_train])
        samples_weight = torch.from_numpy(samples_weight).double()
        sampler = WeightedRandomSampler(samples_weight, num_samples=len(samples_w

        train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, sampler=sample
        val_loader   = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, n
        test_loader  = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False,

        sample_data = train_ds.get(0)
        model, history, best_val_auc = train_onecycle(
            train_loader, val_loader, sample_data, desc_dim=6
        )

        all_histories.append(history)

        crit = focal_loss
        val_metrics, val_true, val_pred = run_validation(val_loader, model, crit)
```

```
        test_metrics, test_true, test_pred = run_validation(test_loader, model, c

        fold_result = {
            "fold": fold,
            "val_accuracy": val_metrics["accuracy"],
            "val_precision": val_metrics["precision"],
            "val_recall": val_metrics["recall"],
            "val_f1": val_metrics["f1"],
            "val_roc_auc": val_metrics["roc_auc"],
            "test_accuracy": test_metrics["accuracy"],
            "test_precision": test_metrics["precision"],
            "test_recall": test_metrics["recall"],
            "test_f1": test_metrics["f1"],
            "test_roc_auc": test_metrics["roc_auc"]
        }
        fold_results.append(fold_result)

        print(f"\nFold {fold} Validation Results:")
        print({k: round(v, 4) for k, v in fold_result.items() if k.startswith('va

        print(f"Fold {fold} Test Results:")
        print({k: round(v, 4) for k, v in fold_result.items() if k.startswith('te

    results_df = pd.DataFrame(fold_results)

    summary = {}
    test_metric_cols = ["test_accuracy", "test_precision", "test_recall", "test_f

    for col in test_metric_cols:
        summary[col] = {
            "mean": results_df[col].mean(),
            "std": results_df[col].std(ddof=1),
            "var": results_df[col].var(ddof=1)
        }

    print("\n" + "="*60)
    print("5-FOLD CROSS-VALIDATION TEST SONUÇLARI")
    print("="*60)
    for col in test_metric_cols:
        print(
            f"{col}: "
            f"{summary[col]['mean']:.4f} ± {summary[col]['std']:.4f} "
            f"(var={summary[col]['var']:.6f})"
        )

    return all_histories, results_df, summary
```

```
import numpy as np
import matplotlib.pyplot as plt

def pad_to_length(lst, target_len):
    if len(lst) >= target_len:
        return lst[:target_len]
    return lst + [lst[-1]] * (target_len - len(lst))

def plot_avg_training_history(histories, max_epoch=35):
    """
    histories: all_histories listesi. Her eleman bir fold'a ait history sözlüğüdü
    max_epoch: her fold'un kullanılacak epoch sayısı
```

```
    """
    tr_auc_matrix = []
    va_auc_matrix = []
    tr_ls_matrix  = []
    va_ls_matrix  = []

    for h in histories:
        tr_auc_matrix.append(pad_to_length(h['tr_auc'], max_epoch))
        va_auc_matrix.append(pad_to_length(h['va_auc'], max_epoch))
        tr_ls_matrix.append(pad_to_length(h['tr_ls'], max_epoch))
        va_ls_matrix.append(pad_to_length(h['va_ls'], max_epoch))

    # Ortalama hesaplamaları
    avg_tr_auc = np.mean(tr_auc_matrix, axis=0)
    avg_va_auc = np.mean(va_auc_matrix, axis=0)
    avg_tr_ls  = np.mean(tr_ls_matrix, axis=0)
    avg_va_ls  = np.mean(va_ls_matrix, axis=0)

    epochs = range(1, max_epoch + 1)

    # AUC Grafiği
    plt.figure(figsize=(8, 5))
    plt.plot(epochs, avg_tr_auc, label='Avg Train AUC', alpha=0.8, marker='o')
    plt.plot(epochs, avg_va_auc, label='Avg Val AUC', alpha=0.8, marker='s')
    plt.xlabel('Epoch')
    plt.ylabel('AUC')
    plt.title(f'Train vs. Validation AUC with Earlystopping')
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()

    # Loss Grafiği
    plt.figure(figsize=(8, 5))
    plt.plot(epochs, avg_tr_ls, label='Avg Train Loss', alpha=0.8, marker='o')
    plt.plot(epochs, avg_va_ls, label='Avg Val Loss', alpha=0.8, marker='s')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title(f'Train vs. Validation Loss with Earlystopping')
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()
```

```
# ----------------------- 9. ANA BLOK -----------------------
if __name__ == "__main__":
    df_all = pd.read_csv("https://raw.githubusercontent.com/McahitKutsal/hivcsv/m
    df_all = df_all.sample(frac=1.0, random_state=SEED).reset_index(drop=True)

    all_histories, results_df, summary = cross_validate_training(df_all, k_folds=

    print("\nFold bazlı sonuçlar:")
    display(results_df)

    results_df.to_csv("GDL_fold_results.csv", index=False)

    summary_rows = []
    for metric_name, vals in summary.items():
        summary_rows.append({
            "Metric": metric_name.replace("test_", "").upper(),
```

```
            "Mean": vals["mean"],
            "Std": vals["std"],
            "Variance": vals["var"],
            "Formatted": f"{vals['mean']:.4f} ± {vals['std']:.4f}"
        })

    summary_df = pd.DataFrame(summary_rows)
    print("\nÖzet tablo:")
    display(summary_df)

    summary_df.to_csv("GDL_summary_results.csv", index=False)
```

```
======================== FOLD 1 ========================
Train: 2308, Val: 288, Test: 289
Epoch 01  Loss: 0.0733/0.0688  AUC: 0.759/0.921  Gap: -0.162
Epoch 02  Loss: 0.0735/0.0647  AUC: 0.773/0.923  Gap: -0.150
Epoch 03  Loss: 0.0679/0.0607  AUC: 0.809/0.935  Gap: -0.126
Epoch 04  Loss: 0.0649/0.0583  AUC: 0.814/0.940  Gap: -0.125
Epoch 05  Loss: 0.0590/0.0538  AUC: 0.852/0.941  Gap: -0.088
Epoch 06  Loss: 0.0558/0.0504  AUC: 0.857/0.939  Gap: -0.082
Epoch 07  Loss: 0.0515/0.0483  AUC: 0.884/0.939  Gap: -0.055
Epoch 08  Loss: 0.0496/0.0467  AUC: 0.880/0.935  Gap: -0.055
Epoch 09  Loss: 0.0464/0.0438  AUC: 0.896/0.938  Gap: -0.042
Epoch 10  Loss: 0.0481/0.0429  AUC: 0.878/0.940  Gap: -0.062
Epoch 11  Loss: 0.0443/0.0434  AUC: 0.899/0.942  Gap: -0.042
Epoch 12  Loss: 0.0435/0.0410  AUC: 0.902/0.945  Gap: -0.043
Epoch 13  Loss: 0.0437/0.0388  AUC: 0.899/0.948  Gap: -0.049
Epoch 14  Loss: 0.0419/0.0403  AUC: 0.914/0.948  Gap: -0.034
Epoch 15  Loss: 0.0394/0.0375  AUC: 0.922/0.951  Gap: -0.029
Epoch 16  Loss: 0.0394/0.0368  AUC: 0.916/0.953  Gap: -0.037
Epoch 17  Loss: 0.0383/0.0338  AUC: 0.923/0.957  Gap: -0.033
Epoch 18  Loss: 0.0377/0.0352  AUC: 0.921/0.956  Gap: -0.036
Epoch 19  Loss: 0.0381/0.0334  AUC: 0.922/0.958  Gap: -0.036
Epoch 20  Loss: 0.0362/0.0338  AUC: 0.924/0.958  Gap: -0.034
Epoch 21  Loss: 0.0337/0.0326  AUC: 0.931/0.959  Gap: -0.028
Epoch 22  Loss: 0.0348/0.0345  AUC: 0.927/0.960  Gap: -0.032
Epoch 23  Loss: 0.0323/0.0356  AUC: 0.940/0.959  Gap: -0.019
Epoch 24  Loss: 0.0332/0.0314  AUC: 0.933/0.961  Gap: -0.028
Epoch 25  Loss: 0.0334/0.0281  AUC: 0.933/0.960  Gap: -0.027
Epoch 26  Loss: 0.0316/0.0279  AUC: 0.937/0.960  Gap: -0.023
Epoch 27  Loss: 0.0312/0.0270  AUC: 0.946/0.961  Gap: -0.014
Epoch 28  Loss: 0.0311/0.0269  AUC: 0.938/0.962  Gap: -0.024
Epoch 29  Loss: 0.0316/0.0309  AUC: 0.930/0.964  Gap: -0.034
Epoch 30  Loss: 0.0296/0.0284  AUC: 0.944/0.964  Gap: -0.020
Epoch 31  Loss: 0.0294/0.0273  AUC: 0.944/0.963  Gap: -0.019
Epoch 32  Loss: 0.0279/0.0273  AUC: 0.950/0.965  Gap: -0.015
Epoch 33  Loss: 0.0296/0.0262  AUC: 0.945/0.962  Gap: -0.017
Epoch 34  Loss: 0.0295/0.0286  AUC: 0.941/0.964  Gap: -0.024
Epoch 35  Loss: 0.0289/0.0285  AUC: 0.941/0.963  Gap: -0.021
Epoch 36  Loss: 0.0287/0.0262  AUC: 0.940/0.963  Gap: -0.023
Epoch 37  Loss: 0.0285/0.0252  AUC: 0.940/0.964  Gap: -0.024
Epoch 38  Loss: 0.0297/0.0247  AUC: 0.940/0.965  Gap: -0.025
Epoch 39  Loss: 0.0286/0.0253  AUC: 0.946/0.966  Gap: -0.020
Epoch 40  Loss: 0.0281/0.0228  AUC: 0.943/0.965  Gap: -0.022
Epoch 41  Loss: 0.0296/0.0216  AUC: 0.941/0.964  Gap: -0.023
Epoch 42  Loss: 0.0273/0.0238  AUC: 0.948/0.966  Gap: -0.018
Epoch 43  Loss: 0.0275/0.0244  AUC: 0.951/0.965  Gap: -0.015
Epoch 44  Loss: 0.0311/0.0231  AUC: 0.935/0.965  Gap: -0.030
Epoch 45  Loss: 0.0266/0.0305  AUC: 0.949/0.965  Gap: -0.016
Epoch 46  Loss: 0.0273/0.0243  AUC: 0.949/0.965  Gap: -0.016
Epoch 47  Loss: 0.0249/0.0271  AUC: 0.955/0.965  Gap: -0.010
Early stopping tetiklendi.

Fold 1 Validation Results:
{'val_accuracy': 0.8646, 'val_precision': 1.0, 'val_recall': 0.7468,
Fold 1 Test Results:
{'test_accuracy': 0.8581, 'test_precision': 1.0, 'test_recall': 0.707

======================== FOLD 2 ========================
Train: 2308, Val: 288, Test: 289
Epoch 01  Loss: 0.1088/0.0894  AUC: 0.689/0.883  Gap: -0.194
```

```
Epoch 02  Loss: 0.1073/0.0878  AUC: 0.700/0.877  Gap: -0.177
Epoch 03  Loss: 0.1014/0.0817  AUC: 0.750/0.918  Gap: -0.168
Epoch 04  Loss: 0.0906/0.0803  AUC: 0.757/0.921  Gap: -0.163
Epoch 05  Loss: 0.0811/0.0739  AUC: 0.830/0.923  Gap: -0.093
Epoch 06  Loss: 0.0684/0.0660  AUC: 0.848/0.923  Gap: -0.074
Epoch 07  Loss: 0.0646/0.0597  AUC: 0.879/0.923  Gap: -0.044
Epoch 08  Loss: 0.0581/0.0542  AUC: 0.876/0.924  Gap: -0.048
Epoch 09  Loss: 0.0555/0.0518  AUC: 0.899/0.924  Gap: -0.026
Epoch 10  Loss: 0.0499/0.0478  AUC: 0.909/0.926  Gap: -0.017
Epoch 11  Loss: 0.0472/0.0432  AUC: 0.911/0.926  Gap: -0.015
Epoch 12  Loss: 0.0435/0.0410  AUC: 0.916/0.927  Gap: -0.011
Epoch 13  Loss: 0.0438/0.0395  AUC: 0.913/0.927  Gap: -0.014
Epoch 14  Loss: 0.0435/0.0374  AUC: 0.917/0.928  Gap: -0.011
Epoch 15  Loss: 0.0401/0.0389  AUC: 0.925/0.927  Gap: -0.002
Epoch 16  Loss: 0.0375/0.0372  AUC: 0.927/0.928  Gap: -0.001
Epoch 17  Loss: 0.0358/0.0365  AUC: 0.936/0.924  Gap: 0.012
Epoch 18  Loss: 0.0346/0.0352  AUC: 0.942/0.927  Gap: 0.015
Epoch 19  Loss: 0.0336/0.0356  AUC: 0.943/0.926  Gap: 0.017
Epoch 20  Loss: 0.0344/0.0349  AUC: 0.936/0.925  Gap: 0.011
Epoch 21  Loss: 0.0329/0.0333  AUC: 0.938/0.923  Gap: 0.015
Epoch 22  Loss: 0.0333/0.0345  AUC: 0.937/0.926  Gap: 0.011
Early stopping tetiklendi.

Fold 2 Validation Results:
{'val_accuracy': 0.8229, 'val_precision': 0.9889, 'val_recall': 0.640
Fold 2 Test Results:
{'test_accuracy': 0.8339, 'test_precision': 1.0, 'test_recall': 0.690

======================== FOLD 3 ========================
Train: 2308, Val: 288, Test: 289
Epoch 01  Loss: 0.1189/0.1039  AUC: 0.442/0.482  Gap: -0.040
Epoch 02  Loss: 0.1180/0.0969  AUC: 0.441/0.732  Gap: -0.291
Epoch 03  Loss: 0.1086/0.0930  AUC: 0.503/0.876  Gap: -0.373
Epoch 04  Loss: 0.0929/0.0769  AUC: 0.617/0.901  Gap: -0.284
Epoch 05  Loss: 0.0793/0.0695  AUC: 0.725/0.903  Gap: -0.177
Epoch 06  Loss: 0.0699/0.0564  AUC: 0.810/0.905  Gap: -0.095
Epoch 07  Loss: 0.0616/0.0489  AUC: 0.842/0.907  Gap: -0.065
Epoch 08  Loss: 0.0546/0.0427  AUC: 0.871/0.910  Gap: -0.039
Epoch 09  Loss: 0.0503/0.0407  AUC: 0.891/0.915  Gap: -0.025
Epoch 10  Loss: 0.0461/0.0393  AUC: 0.900/0.916  Gap: -0.017
Epoch 11  Loss: 0.0448/0.0371  AUC: 0.906/0.916  Gap: -0.010
Epoch 12  Loss: 0.0434/0.0365  AUC: 0.914/0.919  Gap: -0.005
Epoch 13  Loss: 0.0401/0.0377  AUC: 0.912/0.918  Gap: -0.006
Epoch 14  Loss: 0.0387/0.0358  AUC: 0.918/0.918  Gap: 0.000
Epoch 15  Loss: 0.0379/0.0335  AUC: 0.918/0.921  Gap: -0.003
Epoch 16  Loss: 0.0363/0.0364  AUC: 0.926/0.919  Gap: 0.007
Epoch 17  Loss: 0.0354/0.0316  AUC: 0.928/0.921  Gap: 0.007
Epoch 18  Loss: 0.0336/0.0336  AUC: 0.928/0.923  Gap: 0.006
Epoch 19  Loss: 0.0339/0.0342  AUC: 0.929/0.920  Gap: 0.009
Epoch 20  Loss: 0.0320/0.0327  AUC: 0.938/0.921  Gap: 0.017
Epoch 21  Loss: 0.0321/0.0327  AUC: 0.937/0.923  Gap: 0.014
Epoch 22  Loss: 0.0321/0.0313  AUC: 0.932/0.931  Gap: 0.001
Epoch 23  Loss: 0.0324/0.0327  AUC: 0.934/0.927  Gap: 0.007
Epoch 24  Loss: 0.0304/0.0353  AUC: 0.936/0.932  Gap: 0.004
Epoch 25  Loss: 0.0302/0.0318  AUC: 0.941/0.925  Gap: 0.016
Epoch 26  Loss: 0.0301/0.0308  AUC: 0.939/0.932  Gap: 0.007
Epoch 27  Loss: 0.0323/0.0322  AUC: 0.934/0.933  Gap: 0.002
Epoch 28  Loss: 0.0311/0.0371  AUC: 0.933/0.927  Gap: 0.005
Epoch 29  Loss: 0.0294/0.0293  AUC: 0.941/0.934  Gap: 0.007
Epoch 30  Loss: 0.0265/0.0290  AUC: 0.953/0.933  Gap: 0.020
Epoch 31  Loss: 0.0282/0.0301  AUC: 0.940/0.938  Gap: 0.002
```

```
Epoch 31  Loss: 0.0302/0.0301  AUC: 0.940/0.938  Gap: 0.002
Epoch 32  Loss: 0.0288/0.0360  AUC: 0.943/0.937  Gap: 0.005
Epoch 33  Loss: 0.0275/0.0309  AUC: 0.950/0.939  Gap: 0.011
Epoch 34  Loss: 0.0306/0.0315  AUC: 0.938/0.937  Gap: 0.001
Epoch 35  Loss: 0.0302/0.0294  AUC: 0.934/0.942  Gap: -0.008
Epoch 36  Loss: 0.0295/0.0313  AUC: 0.941/0.939  Gap: 0.001
Epoch 37  Loss: 0.0276/0.0324  AUC: 0.946/0.941  Gap: 0.006
Epoch 38  Loss: 0.0291/0.0398  AUC: 0.941/0.939  Gap: 0.003
Epoch 39  Loss: 0.0276/0.0372  AUC: 0.945/0.939  Gap: 0.006
Epoch 40  Loss: 0.0264/0.0312  AUC: 0.950/0.940  Gap: 0.010
Epoch 41  Loss: 0.0282/0.0341  AUC: 0.940/0.938  Gap: 0.002
Epoch 42  Loss: 0.0297/0.0300  AUC: 0.937/0.940  Gap: -0.003
Epoch 43  Loss: 0.0279/0.0349  AUC: 0.944/0.939  Gap: 0.005
Early stopping tetiklendi.

Fold 3 Validation Results:
{'val_accuracy': 0.875, 'val_precision': 1.0, 'val_recall': 0.7722, '
Fold 3 Test Results:
{'test_accuracy': 0.8789, 'test_precision': 1.0, 'test_recall': 0.744

======================== FOLD 4 ========================
Train: 2308, Val: 288, Test: 289
Epoch 01  Loss: 0.1002/0.0864  AUC: 0.578/0.497  Gap: 0.082
Epoch 02  Loss: 0.1037/0.0852  AUC: 0.552/0.691  Gap: -0.139
Epoch 03  Loss: 0.0906/0.0750  AUC: 0.663/0.874  Gap: -0.211
Epoch 04  Loss: 0.0780/0.0691  AUC: 0.747/0.896  Gap: -0.148
Epoch 05  Loss: 0.0684/0.0576  AUC: 0.806/0.914  Gap: -0.107
Epoch 06  Loss: 0.0605/0.0505  AUC: 0.858/0.923  Gap: -0.065
Epoch 07  Loss: 0.0526/0.0458  AUC: 0.877/0.926  Gap: -0.049
Epoch 08  Loss: 0.0502/0.0432  AUC: 0.889/0.928  Gap: -0.039
Epoch 09  Loss: 0.0472/0.0413  AUC: 0.900/0.928  Gap: -0.028
Epoch 10  Loss: 0.0466/0.0406  AUC: 0.891/0.929  Gap: -0.038
Epoch 11  Loss: 0.0408/0.0380  AUC: 0.915/0.930  Gap: -0.016
Epoch 12  Loss: 0.0421/0.0383  AUC: 0.908/0.931  Gap: -0.023
Epoch 13  Loss: 0.0379/0.0361  AUC: 0.931/0.933  Gap: -0.002
Epoch 14  Loss: 0.0374/0.0345  AUC: 0.920/0.937  Gap: -0.018
Epoch 15  Loss: 0.0365/0.0351  AUC: 0.921/0.935  Gap: -0.014
Epoch 16  Loss: 0.0358/0.0354  AUC: 0.929/0.936  Gap: -0.007
Epoch 17  Loss: 0.0365/0.0335  AUC: 0.918/0.938  Gap: -0.020
Epoch 18  Loss: 0.0355/0.0370  AUC: 0.923/0.933  Gap: -0.010
Epoch 19  Loss: 0.0333/0.0323  AUC: 0.931/0.941  Gap: -0.010
Epoch 20  Loss: 0.0329/0.0294  AUC: 0.936/0.943  Gap: -0.007
Epoch 21  Loss: 0.0311/0.0317  AUC: 0.940/0.941  Gap: -0.001
Epoch 22  Loss: 0.0317/0.0330  AUC: 0.936/0.944  Gap: -0.008
Epoch 23  Loss: 0.0328/0.0308  AUC: 0.933/0.947  Gap: -0.014
Epoch 24  Loss: 0.0301/0.0274  AUC: 0.945/0.946  Gap: -0.001
Epoch 25  Loss: 0.0292/0.0305  AUC: 0.942/0.946  Gap: -0.004
Epoch 26  Loss: 0.0308/0.0322  AUC: 0.936/0.944  Gap: -0.008
Epoch 27  Loss: 0.0297/0.0281  AUC: 0.939/0.947  Gap: -0.007
Epoch 28  Loss: 0.0301/0.0274  AUC: 0.936/0.949  Gap: -0.012
Epoch 29  Loss: 0.0283/0.0290  AUC: 0.944/0.947  Gap: -0.003
Epoch 30  Loss: 0.0293/0.0308  AUC: 0.940/0.947  Gap: -0.007
Epoch 31  Loss: 0.0305/0.0275  AUC: 0.934/0.951  Gap: -0.017
Epoch 32  Loss: 0.0264/0.0340  AUC: 0.952/0.947  Gap: 0.005
Epoch 33  Loss: 0.0272/0.0381  AUC: 0.946/0.945  Gap: 0.001
Epoch 34  Loss: 0.0276/0.0362  AUC: 0.944/0.946  Gap: -0.002
Epoch 35  Loss: 0.0266/0.0334  AUC: 0.948/0.947  Gap: 0.001
Epoch 36  Loss: 0.0275/0.0292  AUC: 0.946/0.950  Gap: -0.005
Epoch 37  Loss: 0.0275/0.0299  AUC: 0.946/0.951  Gap: -0.005
Epoch 38  Loss: 0.0255/0.0285  AUC: 0.953/0.949  Gap: 0.004
Epoch 39  Loss: 0.0274/0.0282  AUC: 0.944/0.950  Gap: -0.006
```

```
Early stopping tetiklendi.

Fold 4 Validation Results:
{'val_accuracy': 0.8715, 'val_precision': 0.9913, 'val_recall': 0.76,
Fold 4 Test Results:
{'test_accuracy': 0.8858, 'test_precision': 0.9828, 'test_recall': 0.

========================= FOLD 5 =========================
Train: 2308, Val: 288, Test: 289
Epoch 01  Loss: 0.1233/0.1152  AUC: 0.305/0.330  Gap: -0.025
Epoch 02  Loss: 0.1172/0.1126  AUC: 0.325/0.596  Gap: -0.272
Epoch 03  Loss: 0.1103/0.1062  AUC: 0.380/0.649  Gap: -0.268
Epoch 04  Loss: 0.0943/0.0894  AUC: 0.461/0.899  Gap: -0.437
Epoch 05  Loss: 0.0779/0.0708  AUC: 0.600/0.919  Gap: -0.319
Epoch 06  Loss: 0.0651/0.0534  AUC: 0.739/0.919  Gap: -0.180
Epoch 07  Loss: 0.0546/0.0463  AUC: 0.833/0.924  Gap: -0.091
Epoch 08  Loss: 0.0528/0.0422  AUC: 0.836/0.928  Gap: -0.092
Epoch 09  Loss: 0.0485/0.0397  AUC: 0.866/0.932  Gap: -0.066
Epoch 10  Loss: 0.0435/0.0388  AUC: 0.902/0.933  Gap: -0.031
Epoch 11  Loss: 0.0431/0.0360  AUC: 0.896/0.933  Gap: -0.037
Epoch 12  Loss: 0.0413/0.0394  AUC: 0.902/0.930  Gap: -0.028
Epoch 13  Loss: 0.0387/0.0354  AUC: 0.918/0.931  Gap: -0.013
Epoch 14  Loss: 0.0377/0.0348  AUC: 0.924/0.931  Gap: -0.008
Epoch 15  Loss: 0.0383/0.0329  AUC: 0.920/0.935  Gap: -0.014
Epoch 16  Loss: 0.0372/0.0359  AUC: 0.924/0.933  Gap: -0.008
Epoch 17  Loss: 0.0344/0.0353  AUC: 0.935/0.935  Gap: 0.000
Epoch 18  Loss: 0.0344/0.0338  AUC: 0.933/0.936  Gap: -0.003
Epoch 19  Loss: 0.0337/0.0336  AUC: 0.933/0.933  Gap: 0.000
Epoch 20  Loss: 0.0343/0.0302  AUC: 0.930/0.937  Gap: -0.007
Epoch 21  Loss: 0.0342/0.0307  AUC: 0.929/0.937  Gap: -0.008
Epoch 22  Loss: 0.0315/0.0324  AUC: 0.940/0.935  Gap: 0.005
Epoch 23  Loss: 0.0337/0.0310  AUC: 0.930/0.935  Gap: -0.005
Epoch 24  Loss: 0.0316/0.0308  AUC: 0.939/0.938  Gap: 0.001
Epoch 25  Loss: 0.0328/0.0321  AUC: 0.937/0.933  Gap: 0.003
Epoch 26  Loss: 0.0311/0.0316  AUC: 0.940/0.939  Gap: 0.002
Epoch 27  Loss: 0.0317/0.0293  AUC: 0.941/0.942  Gap: -0.001
Epoch 28  Loss: 0.0323/0.0305  AUC: 0.937/0.937  Gap: 0.001
Epoch 29  Loss: 0.0295/0.0314  AUC: 0.947/0.941  Gap: 0.006
Epoch 30  Loss: 0.0289/0.0315  AUC: 0.945/0.940  Gap: 0.005
Epoch 31  Loss: 0.0290/0.0315  AUC: 0.949/0.939  Gap: 0.010
Epoch 32  Loss: 0.0285/0.0288  AUC: 0.944/0.944  Gap: 0.001
Epoch 33  Loss: 0.0286/0.0340  AUC: 0.949/0.939  Gap: 0.010
Epoch 34  Loss: 0.0291/0.0351  AUC: 0.948/0.939  Gap: 0.008
Epoch 35  Loss: 0.0298/0.0296  AUC: 0.939/0.940  Gap: -0.001
Epoch 36  Loss: 0.0275/0.0321  AUC: 0.951/0.940  Gap: 0.011
Epoch 37  Loss: 0.0298/0.0272  AUC: 0.944/0.938  Gap: 0.005
Epoch 38  Loss: 0.0283/0.0299  AUC: 0.950/0.941  Gap: 0.009
Epoch 39  Loss: 0.0279/0.0289  AUC: 0.950/0.940  Gap: 0.011
Epoch 40  Loss: 0.0271/0.0342  AUC: 0.954/0.938  Gap: 0.016
Early stopping tetiklendi.

Fold 5 Validation Results:
{'val_accuracy': 0.8889, 'val_precision': 1.0, 'val_recall': 0.773, '
Fold 5 Test Results:
{'test_accuracy': 0.8997, 'test_precision': 1.0, 'test_recall': 0.811

============================================================
5-FOLD CROSS-VALIDATION TEST SONUÇLARI
============================================================
test_accuracy: 0.8713 ± 0.0257 (var=0.000661)
```

```
test_precision: 0.9966 ± 0.0077 (var=0.000059)
```

```
plot_avg_training_history(all_histories, max_epoch=35)
```

```
test_roc_auc: 0.9560 ± 0.0154 (var=0.000238)
```

